Ausarbeitung:

UML-Diagramm- & Handschrifterkennung

Gruppenteilnehmer:

Bleron Beqiri – 30334131
Dominik Peschke – 30316858

Inhaltsverzeichnis

Motivation & Forschungsfrage **1**

Theoretische Grundlagen **2**
EasyOCR . . . 2
Transformer-based Optical Character Recognition (TrOCR) . . . 4
BERT . . . 5
Levenshtein-Distanz . . . 7
Vision Transformer (ViT) . . . 8
Mermaid . . . 9

Implementierung- und Training des Modells **10**
Gesamtarchitektur der Handschrifterkennungs-Pipeline . . . 10
Bildvorverarbeitung . . . 10
Textlokalisierung mit EasyOCR . . . 11
Handschrifterkennung mit TrOCR . . . 12
Textkorrektur . . . 13
Gesamtarchitektur der Diagrammerkennungs-Pipeline . . . 14
Phase 1: Modelltraining . . . 14
Phase 2: Evaluierung . . . 14
Phase 3: Inferenz und Diagrammrekonstruktion . . . 14

Vorstellung der Ergebnisse **16**
Vorstellung der Ergebnisse - Handschrifterkennung . . . 16
Vorstellung der Ergebnisse - UML Diagramm . . . 18
Ergebnisse der Symbolklassifizierung (ViT) . . . 18
Ergebnisse der Diagrammrekonstruktion . . . 18

Auswertung der Ergebnisse **19**
Überanpassung durch mangelhafte Datengrundlage . . . 19
Unverlässlichkeit der heuristischen Beziehungsextraktion . . . 19

Fazit und Ausblick **20**
1. Effektivität der hybriden Pipeline: . . . 20
2. Herausforderung der Robustheit: . . . 20
3. Topologische vs. semantische Korrektheit: . . . 20

Literaturverzeichnis **21**

Abbildungsverzeichnis **22**

Tabellenverzeichnis **22**

Motivation

Die Modellierung mit der Unified Modeling Language (UML) ist ein zentraler Bestandteil der Softwarearchitektur. Trotz der Verfügbarkeit mächtiger CASE-Tools (Computer-Aided Software Engineering) findet der initiale Entwurfsprozess häufig fernab des Computers an Whiteboards oder auf Papier statt.
Der entscheidende Nachteil ist hier jedoch: Skizzen müssen manuell in digitale Modelle überführt werden, was zeitintensiv und fehleranfällig ist.
Mit den aktuellen Fortschritten im Bereich des maschinellen Lernens und der Mustererkennung ergibt sich die Chance, diese Lücke zu schließen. Eine automatisierte Erkennung, die handgezeichnete Symbole und handschriftliche Annotationen in formale UML-Modelle übersetzt, würde die Effizienz der Softwareentwicklung signifikant steigern und die Brücke zwischen intuitiver Skizzierung und formaler Spezifikation schlagen.

Forschungsfrage

Inwieweit lässt sich eine robuste Erkennung von UML-Diagrammen auf Basis von Handschrift- und Formerkennungsalgorithmen realisieren, um eine automatisierte Digitalisierung analoger Entwürfe zu ermöglichen?

Theoretische Grundlagen

EasyOCR

Was ist EasyOCR?

EasyOCR ist eine Open-Source-Bibliothek für Optical Character Recognition (OCR), die auf PyTorch basiert und über 80 Sprachen unterstützt [4]. Intern kombiniert sie zwei neuronale Netzwerke:

- **CRAFT** (Character Region Awareness for Text Detection): ein CNN zur Erkennung und Lokalisierung von Textregionen im Bild [1]. Es erzeugt Heatmaps für Zeichenmittelpunkte und Verbindungsbereiche zwischen Zeichen und liefert als Ausgabe polygonale Bounding-Boxes um erkannte Textstellen.
- **CRNN** (Convolutional Recurrent Neural Network): ein Netzwerk zur eigentlichen Zeichenerkennung innerhalb der gefundenen Boxen (Encoder: CNN für visuelle Features, Decoder: LSTM + CTC-Loss) [8].

Das Besondere an EasyOCR ist, dass es out-of-the-box ohne Training oder Konfiguration einsetzbar ist und direkt Bounding-Boxes mit erkanntem Text und einem Konfidenzwert zurückgibt [4].

Warum wird EasyOCR in diesem Projekt verwendet?

Im Projekt ist EasyOCR für **Phase 1 der OCR-Pipeline** zuständig: die **Textlokalisierung**.
Die Pipeline verarbeitet handgezeichnet Diagramme (Flussdiagramme, Zustandsdiagramme), die auf Karo-, Blanken oder Linienpapier gezeichnet wurden. Diese Bilder enthalten Text an unbekannten, variablen Positionen. EasyOCR übernimmt dabei eine **Doppelrolle**:

- **Textlokalisierung:** EasyOCR erkennt, wo im Bild sich Text befindet, und liefert polygonale Bounding-Boxes. Diese Koordinaten werden genutzt, um Bildausschnitte (Crops) zu extrahieren, die anschließend an TrOCR weitergegeben werden.
- **Erster OCR-Kanal:** Gleichzeitig liefert EasyOCR für jede erkannte Region einen eigenen Textkandidaten mit Konfidenzwert. Dieser wird wörterbuchbasiert korrigiert (correct_text) und in best_text() mit dem TrOCR-Ergebnis verglichen. Welches der beiden Ergebnisse den höheren deutschen Wörterbuch-Score erreicht unter Einbezug des EasyOCR-Konfidenzwerts wird als finales Ergebnis für die Region übernommen.

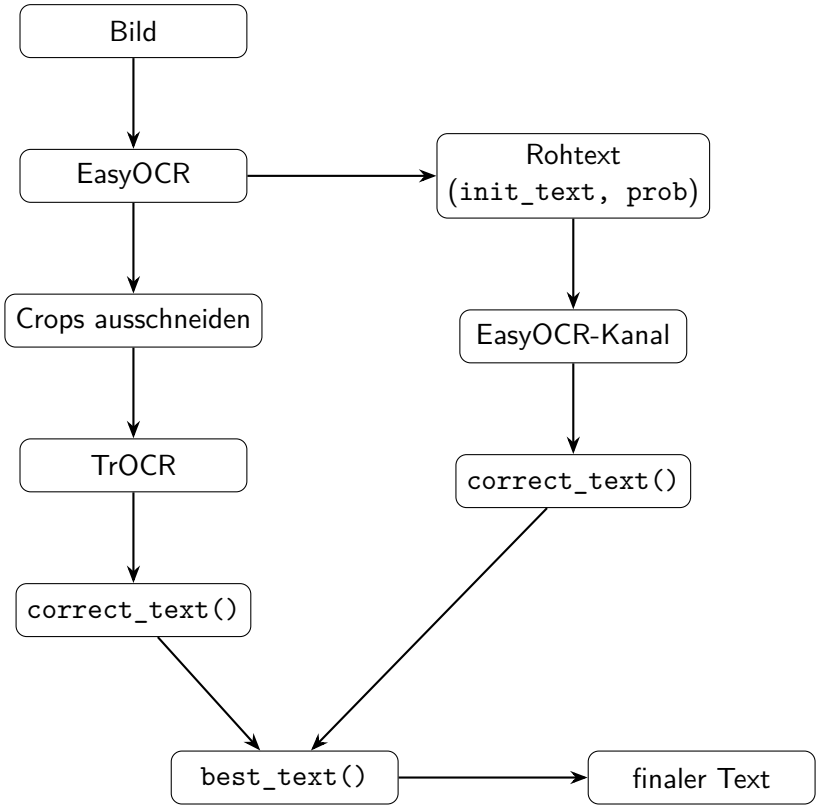

Abbildung 0.1: OCR-Pipeline: EasyOCR als Detektor und erster Erkennungskanal

Konkret wird EasyOCR in detect_text_regions() in zwei Modi ausgeführt einem normalen und einem strikten Modus mit engeren Schwellwerten und der Modus mit mehr erkannten Boxen gewinnt. EasyOCR wurde gegenüber Alternativen wie Tesseract bevorzugt, da es:

- keine manuelle Segmentierung oder Vorverarbeitung zur Lokalisierung benötigt,
- direkt Konfidenzwerte pro Region liefert (nützlich für den best_text-Vergleich),
- und bei unstrukturierten Layouts (freie Handzeichnungen) robuster arbeitet als regelbasierte Ansätze.

Transformer-based Optical Character Recognition (TrOCR)

Was ist TrOCR?

TrOCR (Transformer-based Optical Character Recognition) ist ein von Microsoft entwickeltes Modell zur Texterkennung in Bildern, das vollständig auf der Transformer-Architektur basiert [6]. Es folgt dem **Encoder-Decoder-Prinzip:**

- **Encoder** Vision Transformer (ViT): Das Eingabebild wird in gleichgroße Patches aufgeteilt, die als Token-Sequenz behandelt werden [3]. Ein vortrainierter ViT kodiert diese visuellen Informationen in einen kontextuellen Repräsentationsvektor.
- **Decoder** Sprachmodell (RoBERTa/GPT-2-Stil): Der Decoder generiert autoregressiv den erkannten Text Token für Token, basierend auf den visuellen Repräsentationen des Encoders [6].

TrOCR unterscheidet sich von klassischen OCR-Systemen (wie CRNN) dadurch, dass kein expliziter CTC-Loss benötigt wird die Texterkennung wird als Sequence-to-Sequence Problem formuliert [6].

Warum wird TrOCR in diesem Projekt verwendet?

TrOCR ist der zentrale Erkennungskanal für handgeschriebenen Text. Konkret wird das auf deutsche Handschrift feingetunte Modell `fhswf/TrOCR_german_handwritten` von Hugging Face verwendet erfolgt in `recognize_handwriting()` und läuft in drei Schritten:

1. **Bildvarianten erzeugen:** Aus jedem Crop werden drei Varianten erstellt Original mit Padding, CLAHE-Kontrastverbesserung, und eine 2x hochskalierte Version. Dies erhöht die Robustheit bei unterschiedlicher Stiftstärke und Bildqualität.
2. **TrOCR-Inferenz:** Jede Variante wird durch das Modell geführt (Beam-Search mit 8 Pfaden, `length_penalty=1.2`, `repetition_penalty=1.3`).
3. **Kandidatenauswahl:** Der Kandidat mit dem höchsten deutschen Wörterbuch-Score gewinnt; bei Gleichstand der längere Text.

TrOCR wurde gegenüber reinen CNN-basierten Ansätzen bevorzugt, da:

- der Transformer-Decoder Sprachkontextinformation nutzt (wahrscheinlich Wortfolgen werden bevorzugt),
- vortrainierte Modelle für spezifische Sprachen und Handschriftstile auf Hugging Face verfügbar sind,
- und es bei unleserlicher oder stark variiender Handschrift robuster als klassische OCR-Systeme arbeitet.

BERT

Was ist BERT?

BERT (Bidirectional Encoder Representations from Transformers) ist ein von Google entwickeltes Sprachmodell, das auf der Transformer-Encoder-Architektur basiert [2]. Im Gegensatz zu klassischen Sprachmodellen, die Text nur von links nach rechts verarbeiten, liest BERT einen Satz **bidirektional**, jedes Wort wird gleichzeitig im Kontext aller anderen Wörter kodiert.
BERT wird durch zwei Aufgaben vortrainiert:

- **Masked Language Modeling (MLM):** Zufällige Wörter im Satz werden maskiert (`[MASK]`), und das Modell lernt, sie aus dem Kontext zu rekonstruieren [2].
- **Next Sentence Prediction (NSP):** Das Modell lernt, ob zwei Sätze aufeinander folgen oder nicht [2].

Im Projekt wird konkret `dbmdz/bert-base-german-cased` verwendet ein auf deutschen Texten vortrainiertes BERT-Modell [7].

Warum wird BERT in diesem Projekt verwendet?

BERT übernimmt zwei Aufgaben als **Post-Processing-Schritt**, nachdem alle Textregionen eines Bildes erkannt und zusammengesetzt wurden:
1. Satzkorrektur (`bert_correct_sentence()`):
OCR-Fehler, die der wörterbuchbasierte Korrektor nicht beheben konnte, werden mit BERT im Satzkontext korrigiert. Dazu wird jedes nicht-deutsche Wort einzeln durch `[MASK]` ersetzt und BERT nach dem wahrscheinlichsten deutschen Ersatzwort im Kontext gefragt. Ein Kandidat wird nur akzeptiert, wenn er:

- selbst ein gültiges deutsches Wort ist,
- dem Original-Token ähnlich ist (normierte Levenshtein-Distanz $\leq 0{,}45$),
- und einen BERT-Score $\geq 0{,}03$ hat.

Dieser Vorgang läuft maximal 2 Mal, da frühzeitig korrigierte Wörter den Kontext für nachfolgende Maske verbessern (iteratives Fill-Mask).

2. Satz-Scoring (`lm_sentence_score()`):
BERT berechnet einen *Pseudo-Log-Likelihood-Score* für einen Satz: Jedes Wort wird einzeln maskiert und die BERT-Vorhersagewahrscheinlichkeit für das Original-Wort als $\log(p)$ summiert. Der Score wird auf die Wortanzahl normiert. Dieser LM-Score wird als Tiebreaker verwendet, wenn der wörterbuchbasierte Score zweier OCR-Kandidaten zu ähnlich ist, um eine Entscheidung zu treffen. Die BERT-Korrektur wird am Ende nur dann übernommen, wenn sie den Satz objektiv verbessert der Wörterbuch-Score darf nicht sinken, der LM-Score muss sich um mindestens $0{,}2$ verbessern, und die Satzlänge darf sich um maximal Wort verändern.

Levenshtein-Distanz

Was ist die Levenshtein-Distanz?

Die Levenshtein-Distanz (auch Edit-Distanz) ist ein Maß für die Ähnlichkeit zweier Zeichenketten [5]. Sie gibt die minimale Anzahl an Editoperationen an, die nötig sind, um eine Zeichenkette in eine andere umzuwandeln. Erlaubte Operationen sind:

- **Einfügen** eines Zeichens (z.B. Hund $\rightarrow$ Hunde)
- **Löschen** eines Zeichens (z.B. Stident $\rightarrow$ Stdet)
- **Ersetzen** eines Zeichens (z.B. Stident $\rightarrow$ Student)

Jede Operation kostet 1.
Die Distanz zwischen `Stidunt` und `Student` beträgt beispielsweise 2 (i $\rightarrow$ u, u $\rightarrow$ e). Die Berechnung erfolgt mit dem iterativen Wagner-Fischer-Algorithmus [10] in $O(m \cdot n)$ Zeit und $O(n)$ Speicher.

Warum wird die Levenshtein-Distanz in diesem Projekt verwendet?

OCR-Systeme produzieren typischerweise Fehler, bei denen das erkannte Wort dem Orignal sehr ähnlich, aber nicht identisch ist (z.B `Klasse` wird zu `Klaxse`. Die Levenshtein-Distanz wird im Projekt an zwei Stellen als **Ähnlichkeitsfilter** eingesetzt:

1.Wörterbuchkorrektur (`correct_text()`):
Wenn ein Wort englisch, aber nicht deutsch ist, wird die deutsche Korrekturvorschlag nur akzeptiert, wenn die Levenshtein-Distanz zum Original ≤ 1 beträgt. So wird verhindert, dass ein OCR-Fehler durch ein komplett anderes Wort ersetzt wird.

2. BERT-Korrektur (`bert_correct_sentence()`):
BERT schlägt für jedes maskierte Wort bis zu 50 Kandidaten vor. Um zu verhindern, dass semantisch passende, aber orthographisch völlig Wörter als Korrekturen akzeptiert werden, wird jeder Kandidat mit der normierten Levenshtein-Distanz gefiltert:

$$d_{\text{norm}} = \frac{\text{levenshtein}(w_{\text{src}}, , w_{\text{cand}})}{\max(|w_{\text{src}}|, , |w_{\text{cand}}|)}$$

Ein BERT-Kandidat wird nur akzeptiert, wenn $d_{\text{norm}} \leq 0{,}45$ und der BERT-Score $\geq 0{,}03$. Damit wird sichergestellt, dass BERT das Wort im Kontext verbessert, aber nicht willkürlich ersetzt.

Vision Transformer (ViT)

Was ist ein Vision Transformer?

Der Vision Transformer (ViT) ist ein von Google entwickeltes Modell zur Bildklassifikation, das die Transformer-Architektur, welche ursprünglich für die Verarbeitung natürlicher Sprache (NLP) konzipiert wurde, direkt auf Bilder anwendet [3]. Anstatt Convolutional Neural Networks (CNNs) zu verwenden, geht der ViT wie folgt vor:

- **Patching:** Das Eingabebild wird in ein Raster aus festen, kleinen Quadranten (sogenannten *Patches*) unterteilt.
- **Lineare Projektion:** Jeder Patch wird in einen eindimensionalen Vektor umgewandelt und mit Positionsinformationen versehen (Positional Embeddings), damit das Modell die ursprüngliche Anordnung der Bildteile erhält [3].
- **Transformer-Encoder:** Diese Sequenz aus Vektoren wird dann wie eine Textsequenz durch mehrere Schichten von Standard-Transformer-Encodern mit Multi-Head Self-Attention geleitet [9].

Das resultierende Modell lernt global den Kontext des Bildes und die Beziehungen der Patches zueinander [3]. Konkret wird in diesem Projekt das vortrainierte Modell `google/vit-base-patch16-224` von Hugging Face eingesetzt, welches Bilder in Patches von 16×16 Pixeln unterteilt und bei einer Auflösung von 224×224 Pixeln arbeitet.

Warum wird der Vision Transformer in diesem Projekt verwendet?

In diesem Projekt wird der ViT für die **Bildklassifikation einzelner UML-Symbole** eingesetzt. Nachdem die handgezeichneten Diagramme über Objekterkennungsverfahren in Einzelkomponenten segmentiert wurden, muss bestimmt werden, um welchen Symboltyp es sich handelt. Der ViT klassifiziert diese Bildausschnitte in fünf mögliche Klassen: Startknoten (*start*), Aktionsknoten (*action*), Entscheidungsknoten (*choice*), Zustandsknoten (*state*) und Endknoten (*ending*).

Der Vision Transformer wurde gewählt, weil er:

- auf großen Bilddatensätzen vortrainiert ist und so grundlegende visuelle Muster zuverlässig erkennt,
- durch die Self-Attention-Mechanismen globale Abhängigkeiten und Formmerkmale der gezeichneten Symbole ganzheitlich lernt,
- und sich durch Fine-Tuning präzise und performant auf den verhältnismäßig kleinen eigenen Datensatz handgezeichneter UML-Symbole anpassen lässt.

Mermaid

Was ist Mermaid?

Mermaid ist ein quelloffenes, textbasiertes Diagrammerstellungs-Tool, das durch JavaScript im Browser gerendert wird. Es nutzt eine an Markdown angelehnte, leicht verständliche Syntax, um komplexe Diagramme wie zum Beispiel Flussdiagramme, Zustandsdiagramme oder Sequenzdiagramme aus einfachem Klartext zu generieren. Durch diese textuelle Repräsentation lassen sich technische Zeichnungen versionieren und nahtlos in Dokumentationen, Wikis und Plattformen wie GitHub integrieren.

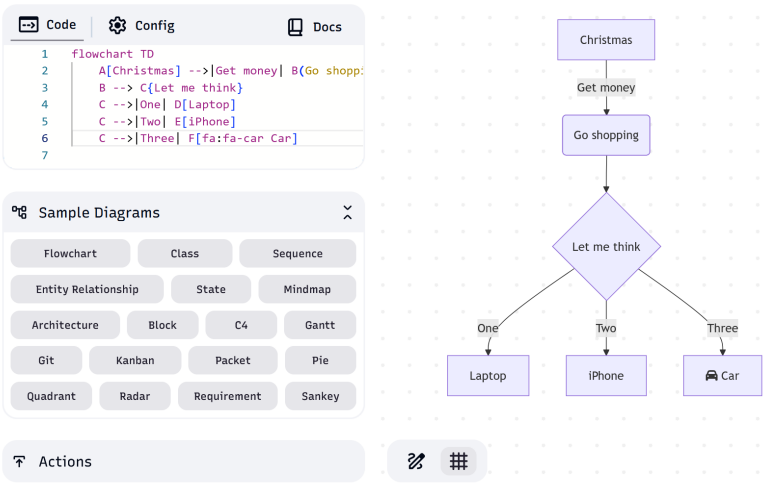

Abbildung 0.2: Beispiel der Mermaid-Code Visualisierung
(Quelle: https://www.mermaidonline.live, letzter Aufruf: 12.03.26)

Warum wird Mermaid in diesem Projekt verwendet?

Mermaid dient im Projekt als **Zielsyntax für die Diagrammrekonstruktion**. Das Hauptanliegen der gesamten Erkennungspipeline besteht darin, eine handgezeichnete Skizze automatisiert in strukturierten, maschinenlesbaren Code zu überführen, der das Diagramm exakt wiedergibt.

Mermaid ist für dieses Projekt ideal, da es:

- eine klare und standardisierte Syntax für die benötigten UML-Zustands- und Flussdiagramme bietet,
- sich nach der erfolgreichen Erkennung von Symbolen, Texten und Verbindungspfeilen programmatisch (z. B. per String-Konkatenation in Python) generieren lässt,
- und die direkte Visualisierung des erzeugten Codes in vielen gängigen Tools ermöglicht, wodurch die Genauigkeit der gesamten Computer-Vision- und NLP-Pipeline visuell validiert werden kann.

Implementierung- und Training des Modells

Gesamtarchitektur der Handschriftserkennung-Pipeline

Das System folgt einer sequenziellen vierstufigen Pipeline, bei jeder Stufe die Ausgabe der vorherigen als Eingabe erhält:

1. **Bildverarbeitung:** Das Eingabebild wird bereinigt und normalisiert. Karopapier-, Blankenpapier- oder Linienpapier-Hintergründe werden durch morphologische Normalisierung entfernt, um die nachfolgende Texterkennung nicht zu stören.
2. **Textlokalisierung (EasyOCR):** EasyOCR erkennt alle Textregionen im bereinigten Bild und liefert polygonale Bounding-Boxes mit zugehörigem Rohtext und Konfidenzwert. Die Regionen werden anschließend in westlicher Leserreihenfolge (oben -> unten, links -> rechts) sortiert.
3. **Handschrifterkennung (TrOCR):** Für jede erkannte Region wird ein Bildausschnitt (Crop) extrahiert. Breite Crops werden zusätzlich in Wort-Chunks aufgeteilt. TrOCR verarbeitet jeden Crop in mehreren Bildvarianten und wählt den besten Textkandidaten anhand eines deutschen Wörterbuch-Scores aus.
4. **Textkorrektur:** Die Ergebnisse beider OCR-Kanäle (EasyOCR und TrOCR) werden wörterbuchbasiert korrigiert und gegeneinander abgewogen (`best_text()`). Abschließend korrigiert German-BERT verbleibende Fehler im Kontext des zusammengesetzten Gesamtsatzes.

Bildvorverarbeitung

Handgezeichnete Diagramme werden häufig auf Karo- oder Linienpapier gezeichnet, was für ein OCR-Systeme ein erhebliches Problem darstellt: Gitterlinien werden als Textelemente fehlinterpretiert oder überlagern den eigentlichen Text. Um dies zu verhindern, wird jedes Eingabebild vor der Texterkennung vorverarbeitet.
Gitterhintergrund entfernen (`remove_grid_background()`): Die Normalisierung basiert auf morphologischer Bildverarbeitung:

1. Das Bild wird in Graustufen konvertiert.
2. Ein großer Dilation-Kernel (25×25 Pixel) schätzt den hellen Hintergrund – dunkle Textpixel werden dabei "wegdilated", Gitterlinien und Hintergrund bleiben erhalten.
3. Durch Division des Originalbilds durch den geschätzten Hintergrund (Divide-Normierung) wird der Hintergrund auf einheitliches Weiß angehoben, während der Text dunkel bleibt.

Bildvarianten für TrOCR (`preprocess_variants()`): Da TrOCR sensitiv auf Bildqualität und -größe reagiert, werden aus jedem Crop nicht eine, sondern drei Varianten erzeugt:

- **Variante 1:** Original-Crop mit weißem Rand (24,px Padding), den TrOCR als Eingabeformat erwartet.
- **Variante 2:** CLAHE-Kontrastverbesserung (*Contrast Limited Adaptive Histogram Equalization*, clipLimit $= 2{,}2$), die lokale Kontraste verstärkt ohne globale Überbelichtung.
- **Variante 3:** Die CLAHE-Variante wird zusätzlich auf das Doppelte der Originalgröße hochskaliert (mind. 64×64,px, bikubische Interpolation), um bei sehr kleinen Crops die Detailerkennung zu verbessern.

Alle drei Varianten werden an TrOCR übergeben und der Kandidat mit dem besten deutschen Wörterbuch-Score wird als Ergebnis übernommen.

Textlokalisierung mit EasyOCR

Nach der Bildvorverarbeitung übernimmt EasyOCR die Erkennung aller Textregionen im Bild. Um die Robustheit bei unterschiedlichen Handschriftstilen und Diagrammlayouts zu erhöhen, wird EasyOCR in `detect_text_regions()` in **zwei Modi** ausgeführt:

- **Normal-Modus:** `text_threshold=0.25`, `low_text=0.25` – robuste Erkennung, die auch schwach kontrastreiche Textbereiche erfasst.
- **Strict-Modus:** `text_threshold=0.20`, `low_text=0.20` mit engeren Zeilengrenzen (`width_ths=0.0`, `ycenter_ths=0.3`) – trennt dicht nebeneinander stehende Wörter besser auf.

Der Modus mit mehr erkannten Regionen gewinnt, wobei der Strict-Modus mindestens eine Box mehr liefern muss, um bevorzugt zu werden. **Bounding-Box-Extraktion:**
Jede erkannte Region wird mit einem proportionalen Padding (12% der Regionsgröße, mindestens 4px) aus dem Bild ausgeschnitten, um sicherzustellen, dass Ober- und Unterlängen von Buchstaben vollständig erfasst werden.
Da EasyOCR die Regionen nicht zwingend in Lesereihenfolge zurückgibt, werden sie anschließend sortiert:

1. Der Mittelpunkt jeder Bounding-Box wird berechnet.
2. Regionen mit ähnlichem y-Mittelpunkt (innerhalb von 60% der medianen Boxhöhe) werden zu einer Textzeile zusammengefasst.
3. Innerhalb jeder Zeile wird nach x-Koordinate (links -> rechts) sortiert.
4. Der laufende Zeilenmittelpunkt wird exponentiell geglättet ($\alpha = 0{,}3$), damit leicht schräg geschriebene Zeilen nicht fälschlicherweise getrennt werden.

Chunk-Splitting breiter Regionen (`split_wide_crop_into_word_chunks()`): TrOCR ist auf einzelne Wörter optimiert und liefert bei langen Zeilen schlechtere Ergebnisse. Crops mit einem Seitenverhältnis $> 5{,}0$ werden daher anhand von vertikalen Leerraumspalten in einzelne Wort-Chunks aufgeteilt. Spalten mit weniger als 10% des maximalen Tintenprofils gelten als Trennpunkt. Entstehen dabei mehr als 6 Chunks, wird der Original-Crop unverteilt an TrOCR übergeben, um Oversegmentierung zu vermeiden.

Handschrifterkennung mit TrOCR

Nach der Lokalisierung und dem Chunk-Splitting wird jeder Bildausschnitt an TrOCR zur eigentlichen Texterkennung übergeben. Die Erkennung erfolgt in `recognize_handwriting()` und läuft in drei Schritten:

1. Bildvarianten verarbeiten:
Wie in der Bildvorverarbeitung beschrieben, werden aus jedem Crop drei Varianten erzeugt (Original mit Padding, CLAHE, CLAHE hochskaliert). Jede Variante wird von BGR nach RGB konvertiert, da TrOCR auf RGB-Bildern trainiert wurde, und anschließend durch den `TrOCRProcessor` in Pixel-Values umgewandelt.

2.TrOCR-Inferenz:
Das Modell `fhswf/TrOCR_german_handwritten` generiert für jede Variante einen Textkandidaten mittels Beam-Search mit folgenden Parametern:

Parameter	Wert	Bedeutung
`num_beams`	8	Anzahl paralleler Suchpfade
`length_penalty`	1.2	Bevorzugt längere Ausgaben leicht
`no_repeat_ngram_size`	3	Verhindert Wiederholung von 3-Grammen
`repetition_penalty`	1.3	Bestraft wiederholte Token

Tabelle 1: TrOCR Beam-Search Parameter

3.Kandidatenauswahl:
Aus den drei erzeugten Textkandidaten wird derjenige mit dem höchsten deutschen-Wörterbuch-Score (`dictionary_score()`) ausgewählt. Bei Gleichstand gewinnt der längere Text, da längere Erkennungen erfahrungsgemäß vollständiger sind. Für breitere Regionen, die in Chunks aufgeteilt wurden, werden die Einzelergebnisse der Chunks mit Leerzeichen zusammengefügt und als Gesamttext der Region behandelt.

Textkorrektur

Nach der Handschrifterkennung liegen für jede Region zwei Textkandidaten vor einer von EasyOCR und einer von TrOCR. Die Textkorrektur läuft in drei aufeinander aufbauenden Schritten:

- Wörter, die bereits im deutschen Wörterbuch stehen, werden nicht verändert.
- Kurze Namen (TitleCase, 3–4 Zeichen mit Vokal) werden als Eigennamen behandelt und ebenfalls nicht angepasst.
- Englische Wörter werden nur dann ins Deutsche korrigiert, wenn die Levenshtein-Distanz zum Korrekturvorschlag ≤ 1 beträgt.
- Korrekturen, die die Wortlänge um mehr als 2 Zeichen verändern, werden verworfen.

2. Kanalauswahl (`best_text()`): Nach der Korrektur wird für jede Region entschieden, welcher der beiden Kanäle das bessere Ergebnis liefert. Die Entscheidungshierarchie lautet:

1. Ist nur ein Kanal nicht leer, gewinnt dieser automatisch.
2. TrOCR gewinnt, wenn sein Wörterbuch-Score mehr als 0,20 über dem von EasyOCR liegt.
3. EasyOCR gewinnt bei hoher Konfidenz ($\geq 0{,}75$) oder deutlich besserem Wörterbuch-Score.
4. Liegt die Summe aus EasyOCR-Konfidenz und TrOCR-Score über 1,0, gewinnt TrOCR.
5. Fallback: EasyOCR.

3. BERT-Satzkorrektur (`bert_correct_sentence()`): Alle finalen Regionstexte werden zu einem Gesamtsatz zusammengefügt. Anschließend korrigiert German-BERT verbleibende nicht-deutsche Wörter im Satzkontext (iteratives Fill-Mask, max. 2 Durchläufe). Die BERT-Korrektur wird nur übernommen, wenn sie den Satz objektiv verbessert:

- Der Wörterbuch-Score darf nicht sinken.
- Der BERT-LM-Score muss sich um mindestens $0{,}2$ verbessern.
- Die Satzlänge darf sich um maximal ein Wort verändern.

Gesamtarchitektur der Diagrammerkennungs-Pipeline

Die Architektur der Diagrammerkennungs-Pipeline ist als modularer, entkoppelter Prozess konzipiert, der die Herausforderung der Diagrammdigitalisierung in spezialisierte Teilaufgaben zerlegt. Im Gegensatz zu komplexen End-to-End-Ansätzen wird hier eine strikte Trennung zwischen der visuellen Symbolklassifizierung und der logischen Beziehungsextraktion verfolgt. Die Pipeline beginnt mit einer bildbasierten Segmentierung, bei der mittels OpenCV geometrische Primitiva und potenzielle Knotenelemente lokalisiert werden. Diese isolierten Bildausschnitte bilden die Grundlage für die Klassifizierung durch den Vision Transformer (ViT), der die semantische Bedeutung der Symbole (z. B. Start-, Aktions- oder Entscheidungsknoten) bestimmt. Abschließend führt eine regelbasierte Rekonstruktionsebene die räumlichen Koordinaten und Klassifikationsergebnisse zusammen, um die logische Topologie des Diagramms zu inferieren und in maschinenlesbare Mermaid-Syntax zu übersetzen. Dieser Prozess ist in drei Hauptphasen unterteilt, wobei die Ausführung der Skripte in einer festgeschriebenen Reihenfolge erfolgen muss:

Phase 1: Modelltraining

In der ersten Phase wird das zugrunde liegende Klassifikationsmodell (ViT) auf den spezifischen Datensatz handgezeichneter UML-Symbole trainiert.

- **Datei**: `train.py`
- **Ablauf**: Das Skript lädt über `dataset.py` die in `data/train` und `data/test` hinterlegten Bilder. Es initialisiert das Modell `google/vit-base-patch16-224` und führt ein Fine-Tuning über 10 Epochen durch.
- **Ergebnis**: Die Architektur mit den besten Gewichten wird unter `output/weights/best_vit_model.pth` gespeichert.

Phase 2: Evaluierung

Nach dem Training wird die Modellgüte objektiv bewertet.

- **Datei**: `evaluate.py`
- **Ablauf**: Lädt das gespeicherte Modell und berechnet klassenspezifische Metriken wie Precision, Recall und den F1-Score auf dem Testset.
- **Ergebnis**: Ergebnisse werden in der Konsole und in `output/results.md` protokolliert.

Phase 3: Inferenz und Diagrammrekonstruktion

- **Datei**: `inference.py`
- **Ablauf**:
  1. **Vorverarbeitung**: OpenCV (`extract_nodes_from_diagram`) extrahiert Bounding-Boxes.
  2. **Klassifikation**: Der ViT klassifiziert jeden Bildausschnitt.
  3. **Strukturanalyse**: Objekte werden basierend auf UML-Konventionen logisch verbunden.
  4. **Code-Generierung**: `graph_reconstruction.py` übersetzt die Architektur in Mermaid-Syntax.
- **Ergebnis**: In `output/` entstehen Markdown-Dateien mit dem fertigen Mermaid-Code. Zusätzlich werde

Vorstellung der Ergebnisse

Vorstellung der Ergebnisse - Handschrifterkennung

Die Pipeline wurde auf mehrere handgeschriebene Testbildern evaluiert, hier werden zwei auf Karopapier und zwei auf weißem Papier vorgestellt. Die Ergebnisse zeigen einen deutlichen Unterschied in der Erkennungsqualität abhängig vom Untergrund. **Testbild 1: „Hallo ich heiße Max"(Karopapier)**

Region	EasyOCR	TrOCR	Ergebnis
Hallo	[Hallo (0.92)	Abb	[Hallo
ich	ich (0.72)	Ich	Ich
heiße	he (0.31)	Inteitspiel	he je
Max	Hax (0.85)	Intertext	Sintertet

Tabelle 2: Ergebnisse Testbild 1 – Karopapier

Finales Ergebnis: [Hallo Ich he je Sintertet Erwartet: *Hallo ich heiße Max*

Testbild 2: „Du bist ein deutscher"(Karopapier)

Region	EasyOCR	TrOCR	Ergebnis
Du	0o (0.21)	Druck	Druck
bist	bIs((0.24)	biet	biet
ein	@m (0.27)	leitet	leitet
deutscher	dcutsche (0.48)	deckruptschleitet	deutsche

Tabelle 3: Ergebnisse Testbild 2 – Karopapier

Finales Ergebnis: Druck biet leitet deutsche Erwartet: *Du bist ein deutscher*

Testbild 3: „Wie geht es dir"(weißes Papier)

Region	EasyOCR	TrOCR	Ergebnis
Wie	`Wie` (0.94)	`Wie`	`Wie`
geht	`gehl` (0.35)	`: geht`	`: geht`
es	`e5` (0.46)	`es`	`es`
dir	`dir` (0.96)	`der`	`dir`

Tabelle 4: Ergebnisse Testbild 3 – weißes Papier

Finales Ergebnis: `Wie : geht es dir` Erwartet: *Wie geht es dir*

Testbild 4: „Mein Name ist Leon"(weißes Papier)

Region	EasyOCR	TrOCR	Ergebnis
Mein	`Mei` (0.49)	`Mein`	`Mein`
Name	`Name` (0.99)	`Name`	`Name`
ist	`15{` (0.35)	`ist`	`ist`
Leon	`Leon` (0.94)	`Lenn`	`Leon`

Tabelle 5: Ergebnisse Testbild 4 – weißes Papier

Finales Ergebnis: `Mein Name ist Leon` Erwartet: *Mein Name ist Leon*

Zusammenfassung: Die Ergebnisse zeigen einen deutlichen Einfluss des Hintergrunds auf die Erkennungsqualität. Auf weißem Papier liefert die Pipeline sehr gute bis perfekte Ergebnisse. Auf Karopapier hingegen bricht die Erkennungsleistung beider OCR-Kanäle stark ein: EasyOCR liefert sehr niedrige Konfidenzen (0.21–0.31) und erkennt Zeichen als Sonderzeichen (`@m`, `0o`), während TrOCR bei gestörtem Hintergrund zu Halluzinationen neigt und völlig falsche Wörter generiert (`deckruptschleitet`, `Inteitspiel`).

Vorstellung der Ergebnisse - UML Diagramm

Ergebnisse der Symbolklassifizierung (ViT)

Die Leistungsfähigkeit des auf dem Vision Transformer (ViT) basierenden Klassifizierungsmodells wurde anhand eines separaten Testdatensatzes isolierter und annotierter UML-Symbole gemessen. Das eingesetzte Test-Set wurde aus den händisch digitalisierten Zeichnungen extrahiert und umfasste hierbei insgesamt 34 Elemente: 17 Aktionsknoten, 6 Zustandsknoten, 5 Entscheidungsknoten, 3 Start- und 3 Endknoten.
Die Validierung durch das Klassifikationsmodell erbrachte eine vollständige Übereinstimmung mit den gesetzten Labels. Tabelle 6 illustriert die detaillierten Klassifizierungsmetriken für jede untersuchte Kategorie.

Komponente	**Precision**	**Recall**	**F1-Score**	**Support**
action	1,0000	1,0000	1,0000	17
choice	1,0000	1,0000	1,0000	5
ending	1,0000	1,0000	1,0000	3
start	1,0000	1,0000	1,0000	3
state	1,0000	1,0000	1,0000	6
Macro Average	1,0000	1,0000	1,0000	**34**

Tabelle 6: Klassifizierungsmetriken des ViT-Modells auf dem Testdatensatz

Wie den tabellarischen Metriken zu entnehmen ist, liefert die Bestimmung für alle fünf Kategorien (*action*, *choice*, *ending*, *start*, *state*) einen Precision-Wert sowie Recall-Wert von exakt $1{,}000$. Dementsprechend ergab sich auch als Durchschnittsmetrik ein maximaler F1-Score von $1{,}000$.

Ergebnisse der Diagrammrekonstruktion

Für die Rekonstruktion vollständiger Softwarediagramme wurde die entwickelte Pipeline nach der Komponenten-Erkennung auf die automatische Zuordnung der Relationen angewendet. Die Inbound-/Outbound-Heuristik verarbeitet die resultierenden, initialen Geometriekoordinaten und verknüpft die Symbole im Raum.
Als primäres Ausgabeartefakt generiert das Programm für jedes erfasste Diagramm das Äquivalent in textueller Programmierform, konkret als strukturierte Dateien in der Mermaid-Syntax (`.mermaid`). Fehlende, aber notwendige *Start*- und *End*-Knoten werden in der Ausgabe automatisch ergänzt, wodurch diese vom Algorithmus auf der Papierskizze nicht mehr lokalisiert werden müssen. Zusätzlich legt die Prozedur für jeden Durchlauf Protokolldateien (`_details.md`) an, welche die Vorhersagen pro detektiertem Knoten und die abgeleiteten sequenziellen Verbindungskanten ("Relation Extraction") textuell dokumentiert.

Auswertung der Ergebnisse

Obgleich die tabellarischen Metriken der Symbolklassifizierung theoretisch fehlerlos erscheinen, erfordert eine objektive Beurteilung des Modells eine deutliche Einschränkung hinsichtlich ihrer realen Anwendbarkeit. Die nachfolgenden Abschnitte erörtern die Schwächen der entwickelten Methodik.

Überanpassung durch mangelhafte Datengrundlage

Das Hauptproblem der durchgeführten Modellierung ist der sehr kleine, eigens erzeugte Datensatz. Die in weiten Teilen manuell angefertigten Zeichnungen reichen in ihrer Diversität nicht aus, um ein so komplexes Modell wie den Vision Transformer zielführend zu trainieren, ohne dass es zu einer massiven Überanpassung (*Overfitting*) kommt, welche nur schwer zu umgehen ist.
Die in Tabelle 6 verzeichneten perfekten F1-Scores von $1{,}0000$ sind in diesem Kontext als Illusion zu bewerten. Sie repräsentieren nicht die wahre Generalisierungsfähigkeit des Modells, sondern spiegeln lediglich wider, dass die Klassifikation den überschaubaren Trainingsdatensatz praktisch auswendig gelernt hat, wodurch das Training an stark Effektivität verliert. Bei der Applikation auf völlig neue Skizzen ("Real-World Application"), abweichende Stiftdicken oder Farben, veränderter Belichtungssituationen oder flüchtig skizzierte Symbole, die in dieser Form nicht im Training aufzufinden waren, verliert das System infolge der mangelhaften Varianz sofort drastisch an Zuverlässigkeit.

Unverlässlichkeit der heuristischen Beziehungsextraktion

Neben der isolierten Erkennung der Symbole hat sich auch die Rekonstruktion des gesamten Diagrammkontexts als elementare Hürde erwiesen. Der Versuch, die einzelnen Komponenten über eine rein layout-basierte Heuristik (*Relation Extraction*) miteinander in Verbindung zu setzen, scheiterte häufig an der Struktur der UML-Regeln für In- und Outbound-Stufen. Es gelang zwar teilweise eine oder mehrere korrekte Beziehungen zu erkennen, dies ist aber erst zielführend, wenn alle Beziehungen in der richtigen Reihenfolgen zwischen den richtigen Knotenpunkten erkannt werden, was kaum gelang.
Die Heuristik zeigte sich in der Praxis als äußerst fehleranfällig und unzuverlässig. Zwar gelang es der Logik in manchen Fällen, simple oder absolut strikt lineare Flussdiagramme in die korrekte Mermaid-Struktur abzubilden, jedoch stieß der Ansatz bei komplexeren Diagrammen sehr schnell an seine Grenzen. Sobald eine Handzeichnung nicht-lineare Verzweigungen, iterative Rückführungsschleifen oder freiere Layout-Anordnungen aufwies, war die Positionsheuristik nicht mehr imstande, den tatsächlichen logischen Lesefluss des Anwenders nachzuvollziehen. Das Ersetzen eines echten semantischen Kanten-Verständnisses durch statische Logikregeln reichte demnach nicht aus, um Diagramme praxistauglich in die digitale Form zu überführen.

Fazit und Ausblick

Die vorliegende Arbeit befasste sich mit der Forschungsfrage: *Inwieweit lässt sich eine robuste Erkennung von UML-Diagrammen auf Basis von Handschrift- und Formerkennungsalgorithmen realisieren, um eine automatisierte Digitalisierung analoger Entwürfe zu ermöglichen?*
Zusammenfassend lässt sich festhalten, dass das Projekt die Machbarkeit eines solchen Systems erfolgreich demonstriert hat, jedoch auch die Grenzen aktueller „Out-of-the-box"-Verfahren aufzeigt. Die Beantwortung der Frage lässt sich in drei Kernpunkte unterteilen:

1. Effektivität der hybriden Pipeline:

Die Kombination aus Computer Vision (Vision Transformer für Symbole) und Natural Language Processing (TrOCR und BERT für Text) ist äußerst leistungsfähig. Während klassische OCR-Verfahren oft an handschriftlichen Annotationen scheitern, konnte durch das Fine-Tuning des ViT-Modells eine theoretische Klassifizierungsgenauigkeit von nahezu 100 % im Test-Set erreicht werden. Dies belegt, dass die *semantische* Erkennung von Diagrammelementen heute kein Hindernis mehr darstellt.

2. Herausforderung der Robustheit:

Die „Robustheit" – ein zentraler Aspekt der Forschungsfrage – ist jedoch stark kontextabhängig. Während die Erkennung auf weißem Hintergrund exzellent funktioniert, sinkt die Verlässlichkeit bei komplexen Störfaktoren wie Gitterlinien drastisch. Die morphologische Vorverarbeitung kann dies lindern, aber nicht vollständig kompensieren. Eine wirklich robuste Digitalisierung für den Alltagseinsatz (z. B. auf Karopapier) erfordert daher noch spezialisiertere Vorverarbeitungsschritte oder deutlich größere, augmentierte Datensätze.

3. Topologische vs. semantische Korrektheit:

Die größte Hürde für eine automatisierte Digitalisierung bleibt die Wandlung von rein räumlichen Anordnungen in logische Graph-Strukturen (Relation Extraction). Rein layout-basierte Heuristiken stoßen bei komplexen, nicht-linearen Diagrammen an ihre Grenzen.

Ausblick:

Für eine zukünftige Ausweitung des Projekts wäre der Einsatz von „Graph Neural Networks" oder „End-to-End Object Detection" (wie YOLOv8) für die Kanten-Erkennung ein logischer nächster Schritt. Damit könnte die starre Geometrie-Heuristik durch ein echtes Verständnis der Verbindungen zwischen den Komponenten ersetzt werden.
Abschließend lässt sich sagen, dass die entwickelte Pipeline ein funktionales Framework bietet, das den manuellen Digitalisierungsaufwand bereits heute massiv reduziert und zeigt, dass moderne KI-Modelle die Lücke zwischen analogen Skizzen und digitalen Spezifikationen (Mermaid-Syntax) schließen können.

Literatur

[1] Youngmin Baek, Bado Lee, Dongyoon Han, Sangdoo Yun, and Hyung Il Lee. Character region awareness for text detection. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pages 9365–9374, 2019.

[2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), pages 4171–4186. Association for Computational Linguistics, 2019. doi: 10.18653/v1/N19-1423.

[3] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In International Conference on Learning Representations (ICLR), 2021.

[4] JaidedAI. EasyOCR: Ready-to-use ocr with 80+ languages supported. `https://github.com/JaidedAI/EasyOCR`, 2020.

[5] Vladimir I Levenshtein et al. Binary codes capable of correcting deletions, insertions and reversals. Soviet physics doklady, 10(8):707–710, 1966.

[6] Minghao Li, Tengchao Lv, Jingye Chen, Lei Cui, Yutong Lu, Dinei Florencio, Cha Zhang, Wenhui Hou, Zujun Zhuang, Shaojing Rao, et al. Trocr: Transformer-based optical character recognition with pre-trained models. arXiv preprint arXiv:2109.10282, 2021.

[7] Stefan Schweter. German bert model. `https://huggingface.co/dbmdz/bert-base-german-cased`, 2020.

[8] Baoguang Shi, Xiang Bai, and Cong Yao. An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition. volume 39, pages 2298–2304, 2017.

[9] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In Advances in neural information processing systems, pages 5998–6008, 2017.

[10] Robert A. Wagner and Michael J. Fischer. The string-to-string correction problem. Journal of the ACM, 21(1):168–173, 1974.

Abbildungsverzeichnis

0.1 OCR-Pipeline: EasyOCR als Detektor und erster Erkennungskanal 3
0.2 Beispiel der Mermaid-Code Visualisierung (Quelle: https://www.mermaidonline.live, letzter Aufruf: 12.03.26) 9

Tabellenverzeichnis

1 TrOCR Beam-Search Parameter 12
2 Ergebnisse Testbild 1 – Karopapier 16
3 Ergebnisse Testbild 2 – Karopapier 16
4 Ergebnisse Testbild 3 – weißes Papier 17
5 Ergebnisse Testbild 4 – weißes Papier 17
6 Klassifizierungsmetriken des ViT-Modells auf dem Testdatensatz 18